



网络安全创新创业实践

实验 2

比特币加密算法关于问题修复与性能优化的实例分析

程洪鑫 202300460051 网安 23.1

2026 年 7 月 20 日

目录

1	实验背景	1
2	Lower-S Normalization 修复分析	2
2.1	问题描述	2
2.1.1	ECDSA 签名可塑性	2
2.1.2	交易可塑性	2
2.2	数学原理	2
2.2.1	ECDSA 签名过程	2
2.2.2	ECDSA 验证过程	2
2.2.3	ECDSA 签名可塑性	3
2.3	问题修复	3
2.3.1	新增签名标准化函数	3
2.3.2	修改 ECDSA 验证逻辑	4
3	WNAF 标量乘法优化分析	4
3.1	椭圆曲线标量乘法性能瓶颈	4
3.2	WNAF 数学原理	4
3.3	WNAF 代码实现	5
4	总结	6

1 实验背景

Bitcoin 是一种基于密码学的去中心化数字货币系统。为了保证交易的真实性和不可伪造性，Bitcoin 使用数字签名算法验证交易所有权。

Bitcoin 使用的核心签名算法为 ECDSA, 其底层椭圆曲线为 secp256k1。

Bitcoin Core 使用开源密码库：

<https://github.com/bitcoin-core/secp256k1>

实现高性能的椭圆曲线计算以及 ECDSA 签名验证。

本文针对 bug fix 和 performance improvement 两部分进行分析，各自挑选了一个示例。

本文分析的 bug fix 来自 commit:

Commit: 0c6ab2f

Title: Introduce explicit lower-S normalization

Author: sipa

Date: Oct 21, 2015

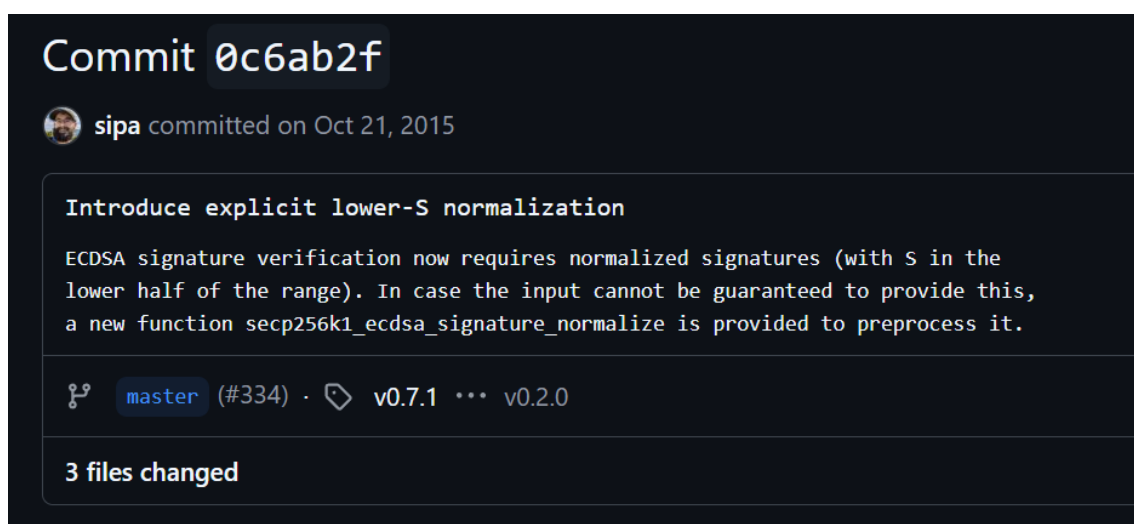


图 1: commit 0c6ab2f

该 commit 修复了 ECDSA 签名可塑性 (ECDSA Signature Malleability) 问题，并引入 `secp256k1_ecdsa_signature_normalize()` 函数，用于将 ECDSA 签名转换为唯一的 Low-S 格式。

本文分析的 performance improvement 来自 `ecmult_impl.h`，是关于 WNAF 标量乘法优化的分析。

2 Lower-S Normalization 修复分析

2.1 问题描述

2.1.1 ECDSA 签名可塑性

ECDSA 签名由两个整数构成: (r, s) 。其中 r 为椭圆曲线点横坐标, s 为签名计算得到的标量。在原来的 secp256k1 实现中, 一个合法签名 (r, s) 同时存在另一个等价形式 $(r, n - s)$, 其中 n 是 secp256k1 曲线生成元 G 的阶。因此, 同一个交易可以存在两个不同的有效签名: $Signature_1 = (r, s)$ 和 $Signature_2 = (r, n - s)$ 。这导致交易数据不同, 但是签名验证均通过。

2.1.2 交易可塑性

Bitcoin 中每一笔交易都有唯一的交易 ID: $TXID = SHA256(SHA256(Transaction))$ 而交易签名属于交易数据的一部分。因此, 如果将签名 (r, s) 替换为 $(r, n - s)$, 虽然交易内容没有改变, 但是交易数据发生变化, 从而导致 $TXID_1 \neq TXID_2$ 。

攻击者可以获取一个已经签名的交易, 修改签名中的 s 从而得到另一个合法签名, 然后广播到网络中, 从而产生新的交易 ID。这样攻击者无需知道私钥, 只需要修改签名即可产生新的合法交易。

这会导致:

- 原交易和修改后的交易同时有效
- 钱包无法准确判断交易状态
- 依赖 TXID 的二层协议受到影响

2.2 数学原理

2.2.1 ECDSA 签名过程

设 d 为私钥。公钥为 $P = dG$, 其中 G 是椭圆曲线生成元。

对于消息 m : $e = Hash(m)$ 。

签名者随机选择 k 。

计算 $R = kG$ 。

得到 $r = x_R \bmod n$ 。

随后计算 $s = k^{-1}(e + rd) \bmod n$ 。

最终签名: (r, s) 。

2.2.2 ECDSA 验证过程

验证者计算 $w = s^{-1} \bmod n$ 。

然后 $u_1 = ew$, $u_2 = rw$ 。

计算 $X = u_1G + u_2P$ 。

如果 $r = x_X \bmod n$, 则签名有效。

2.2.3 ECDSA 签名可塑性

考虑新的签名： $s' = n - s$ 。

由于所有标量运算均在模 n 的有限域中，所以 $s' \equiv -s \pmod{n}$ 。

因此 $(s')^{-1} = (-s)^{-1}$ ，即 $(s')^{-1} = -s^{-1}$ 。

所以新的验证参数： $u'_1 = e(s')^{-1}$ ，即 $u'_1 = -u_1$ 。同理， $u'_2 = -u_2$ 。

新的验证点： $X' = u'_1 G + u'_2 P$ ，即 $X' = -u_1 G - u_2 P$ ，因此 $X' = -X$ 。

在椭圆曲线中，如果 $X = (x, y)$ ，则 $-X = (x, -y)$ ，故 $x_X = x_{-X}$ 。

而 ECDSA 验证只检查 $r = x_X \pmod{n}$ 。因此： (r, s) 和 $(r, n - s)$ 均能通过验证。

2.3 问题修复

2.3.1 新增签名标准化函数

```
259 + int secp256k1_ecdsa_signature_normalize(const secp256k1_context* ctx, secp256k1_ecdsa_signature *sigout, const secp256k1_ecdsa_signature *signin) {
260 +     secp256k1_scalar r, s;
261 +     int ret = 0;
262 +
263 +     VERIFY_CHECK(ctx != NULL);
264 +     ARG_CHECK(signin != NULL);
265 +
266 +     secp256k1_ecdsa_signature_load(ctx, &r, &s, signin);
267 +     ret = secp256k1_scalar_is_high(&s);
268 +     if (sigout != NULL) {
269 +         if (ret) {
270 +             secp256k1_scalar_negate(&s, &s);
271 +         }
272 +         secp256k1_ecdsa_signature_save(sigout, &r, &s);
273 +     }
274 +
275 +     return ret;
276 + }
277 +
```

图 2: secp256k1_ecdsa_signature_normalize()

该 commit 新增函数：`secp256k1_ecdsa_signature_normalize()`，用于将 ECDSA 签名转换为唯一的 Low-S 格式。

该函数完整定义如下：

```
1 int secp256k1_ecdsa_signature_normalize(
2     const secp256k1_context* ctx,
3     secp256k1_ecdsa_signature *sigout,
4     const secp256k1_ecdsa_signature *signin)
```

函数首先读取输入签名中的两个标量：

```
secp256k1_ecdsa_signature_load(ctx, &r, &s, signin);
```

其中， r 表示签名中的随机点横坐标， s 表示签名标量。

随后通过：`secp256k1_scalar_is_high(&s)`，判断当前 s 是否属于 High-S。该判断等价于 $s > n/2$ 。如果 s 过大，则执行：`secp256k1_scalar_negate(&s, &s)`，将 s 取负数。由于标量运算均在模 n 的有限域中进行： $-s \equiv n - s \pmod{n}$ ，因此该操作实际是将 s 替换为 $n - s$ 。

例如：

如果原始签名： (r, s) 满足： $s > n/2$ ，则规范化后变为 $(r, n - s)$ 。最终满足： $s \leq n/2$ 。

因此该函数保证所有合法签名都会被转换为唯一 Low-S 表示。

2.3.2 修改 ECDSA 验证逻辑

```
259 278 int secp256k1_ecdsa_verify(const secp256k1_context* ctx, const secp256k1_ecdsa_signature *sig, const unsigned char *msg32, const secp256k1_pubkey *pubkey) {
260 279     secp256k1_ge q;
261 280     secp256k1_scalar r, s;
262 281     @@ -268,7 +287,8 @@ int secp256k1_ecdsa_verify(const secp256k1_context* ctx, const secp256k1_ecdsa_s
268 287
269 288     secp256k1_scalar_set_b32(&r, msg32, NULL);
270 289     secp256k1_ecdsa_signature_load(ctx, &r, &s, sig);
271 - return (secp256k1_pubkey_load(ctx, &q, pubkey) &&
290 + return (!secp256k1_scalar_is_high(&s) && secp256k1_ecdsa_sig_verify(ctx, &r, &s, &q, pubkey));
291 + secp256k1_pubkey_load(ctx, &q, pubkey) &&
272 292     secp256k1_ecdsa_sig_verify(ctx, &r, &s, &q, pubkey));
```

图 3: secp256k1_ecdsa_sig_verify()

在修复之前，验证函数：secp256k1_ecdsa_sig_verify(...) 会接受所有数学上有效的 ECDSA 签名。也就是说 (r, s) 和 $(r, n - s)$ 均会通过验证。

修改后，验证流程增加 secp256k1_scalar_is_high(&s) 检查。

新的验证逻辑为

```
return (!secp256k1_scalar_is_high(&s) && secp256k1_ecdsa_sig_verify(...));
```

验证过程变为：首先判断 s 是否满足： $s \leq n/2$ 。如果满足，则继续执行标准 ECDSA 验证。

因此，High-S signature 会被直接拒绝。只有满足 $1 \leq s \leq n/2$ 的 Low-S signature 才能被 Bitcoin 节点接受。

3 WNAF 标量乘法优化分析

3.1 椭圆曲线标量乘法性能瓶颈

在 ECDSA 算法中，椭圆曲线标量乘法是最主要的计算操作之一。

例如，在 ECDSA 签名过程中，需要计算 $R = kG$ ，其中 k 为随机数， G 为 secp256k1 曲线生成元；在 ECDSA 验证过程中，需要计算 $X = u_1G + u_2Q$ ，其中 Q 为公钥。因此，椭圆曲线点乘 kP 的计算效率会直接影响 Bitcoin 交易签名和验证性能。

传统方法通常采用二进制展开： $k = \sum_{i=0}^n k_i 2^i$ ，其中 $k_i \in \{0, 1\}$ 。计算过程为： $kP = k_0P + k_1(2P) + k_2(4P) + \dots$ 该方法需要大量的点加法操作。由于椭圆曲线点加法涉及有限域乘法和求逆运算，计算成本较高，因此 secp256k1 引入 WNAF (Window Non-Adjacent Form) 方法优化标量乘法。

3.2 WNAF 数学原理

WNAF (Window Non-Adjacent Form) 的核心思想是将普通二进制表示转换为一种稀疏的带符号数字表示形式。对于任意标量 k ，WNAF 将其表示为： $k = \sum_i d_i 2^i$ ，其中每个系数满足： $d_i \in \{0, \pm 1, \pm 3, \pm 5, \dots, \pm(2^{w-1} - 1)\}$ 。与普通二进制表示不同，WNAF 要求任意两个非零系数之间至少间隔 $w - 1$ 个零，即： $d_i \neq 0 \Rightarrow d_{i+1} = d_{i+2} = \dots = d_{i+w-1} = 0$ 。因此，WNAF 表示具有较高的稀疏性，可以显著减少标量乘法过程中椭圆曲线点加操作的次数。

例如，考虑标量 $k = 15$ 。在普通二进制表示中： $15 = (1111)_2 = 1 + 2 + 4 + 8$ ，因此： $15P = P + 2P + 4P + 8P$ 需要执行多次点加操作。通过 WNAF 转换，可以得到： $15 = 16 - 1 = 2^4 - 1$ ，对应的稀疏表示为： $15 = (-1) \times 2^0 + 1 \times 2^4$ ，即 $15P = -P + 16P$ ，其中非零系数数量由原

来的 4 个减少为 2 个。由于椭圆曲线点加操作的计算成本较高，减少非零系数数量能够降低标量乘法过程中的点加次数。

此外，在窗口大小为 w 时，WNAF 中所有非零系数均为奇数倍点： $P, 3P, 5P, \dots, (2^{w-1}-1)P$ 因此可以提前计算这些点并存储在预计算表中。在实际计算过程中，只需要根据 WNAF 系数 d_i 选择对应的预计算点，从而进一步提高椭圆曲线标量乘法效率。

3.3 WNAF 代码实现

```

162     static int secp256k1_ecmult_wnaf(int *wnaf, int len, const secp256k1_scalar *a, int w) {
163         secp256k1_scalar s;
164         int last_set_bit = -1;
165         int bit = 0;
166         int sign = 1;
167         int carry = 0;
168
169         VERIFY_CHECK(wnaf != NULL);
170         VERIFY_CHECK(0 <= len && len <= 256);
171         VERIFY_CHECK(a != NULL);
172         VERIFY_CHECK(2 <= w && w <= 31);
173
174         for (bit = 0; bit < len; bit++) {
175             wnaf[bit] = 0;
176         }
177
178         s = *a;
179         if (secp256k1_scalar_get_bits_limb32(&s, 255, 1)) {
180             secp256k1_scalar_negate(&s, &s);
181             sign = -1;
182         }
183
184         bit = 0;
185         while (bit < len) {
186             int now;
187             int word;
188             if (secp256k1_scalar_get_bits_limb32(&s, bit, 1) == (unsigned int)carry) {
189                 bit++;
190                 continue;
191             }
192
193             now = w;
194             if (now > len - bit) {
195                 now = len - bit;
196             }
197
198             word = secp256k1_scalar_get_bits_var(&s, bit, now) + carry;
199
200             carry = (word >> (w-1)) & 1;
201             word -= carry << w;
202
203             wnaf[bit] = sign * word;
204             last_set_bit = bit;
205
206             bit += now;
207         }

```

图 4: secp256k1_ecmult_wnaf()

在 secp256k1 中, WNAF 转换通过函数 `secp256k1_ecmult_wnaf()` 实现, 其函数定义如下:

```
1 static int secp256k1_ecmult_wnaf(
2     int *wnaf,
3     int len,
4     const secp256k1_scalar *a,
5     int w)
```

其中, a 表示需要转换的标量 k , $wnaf$ 用于保存生成的 WNAF 数组, w 表示窗口大小。WNAF 转换的核心过程位于 `while (bit < len)` 循环中。

首先, 函数按位扫描标量。如果当前位置不需要放入非零系数, 就直接把该位对应的系数记为 0, 即 $d_i = 0$ 。

当扫描到需要生成非零系数的位置时, 函数读取连续 w 位, 形成一个窗口值:

```
1 word = secp256k1_scalar_get_bits_var(&s, bit, now) + carry;
```

这里的 `carry` 表示前一个窗口处理后留下的进位。

随后, 函数检查窗口值是否超过当前窗口宽度能直接表示的范围:

```
1 carry = (word >> (w-1)) & 1;
2 word -= carry << w;
```

这一步的作用是把窗口值赋值到对称区间中。若 `word` 太大, 就通过设置进位并减去 2^w , 使得系数落在 $\{0, \pm 1, \pm 3, \pm 5, \dots\}$ 集合中。

例如, 当窗口大小为 $w = 4$ 时, 窗口值 $m = 9$ 已经超过了可直接表示的正向范围。此时执行: $9 - 2^4 = 9 - 16 = -7$, 所以对应的 WNAF 系数就是 $d_i = -7$ 。这说明同一个标量可以用“较少的非零奇数项”来表示, 而不是保留很多连续的 1。

最后, 函数把当前窗口得到的系数写入 WNAF 数组:

```
1 wnaf[bit]=sign*word;
```

通过上述过程, secp256k1 将普通二进制标量转换为稀疏的 WNAF 表示, 减少标量乘法中的非零项数量, 从而降低椭圆曲线点加运算次数, 提高 ECDSA 签名和验证效率。

4 总结

本文分析了 Bitcoin secp256k1 项目中的两个示例。

第一个示例是 Lower-S Normalization 修复分析, 解决了 ECDSA signature malleability 问题。问题产生的根源是: ECDSA 验证只依赖椭圆曲线点的 x 坐标, 导致 (r, s) 和 $(r, n - s)$ 具有相同验证结果。通过引入 Low-S normalization, secp256k1 强制 $s \leq n/2$, 使每个签名具有唯一表示。该修改没有改变 ECDSA 的密码学安全性, 但是提高了 Bitcoin 交易系统的一致性和可靠性, 并成为后续 Bitcoin 签名标准的重要基础。

第二个示例是 secp256k1 通过 WNAF 算法优化椭圆曲线标量乘法。该方法将普通二进制表示: $k = \sum k_i 2^i$ 转换为稀疏表示: $k = \sum d_i 2^i$, 减少非零系数数量, 降低椭圆曲线点加次数。同时结合预计算表, 能够快速获得: $P, 3P, 5P, \dots, (2^{w-1} - 1)P$, 进一步提升点乘计算效率。